

Source: Google Cloud Documentation, *MLOps: Continuous Delivery and Automation Pipelines in Machine Learning*

Section: MLOps Level 2: CI/CD pipeline automation

Licence: Creative Commons Attribution 4.0 International (CC BY 4.0)

Original source: Google Cloud Architecture Center

Content reproduced from a larger document and provided for educational use.

Contributors:

Authors:

Jarek Kazmierczak | Solutions Architect

Khalid Salama | Staff Software Engineer, Machine Learning

Valentín Huerta | AI Engineer

Other contributor:

Sunil Kumar Jang Bahadur | Customer Engineer

MLOps level 2: CI/CD pipeline automation

For a rapid and reliable update of the pipelines in production, you need a robust automated CI/CD system. This automated CI/CD system lets your data scientists rapidly explore new ideas around feature engineering, model architecture, and hyperparameters. They can implement these ideas and automatically build, test, and deploy the new pipeline components to the target environment.

The following diagram shows the implementation of the ML pipeline using CI/CD, which has the characteristics of the automated ML pipelines setup plus the automated CI/CD routines.

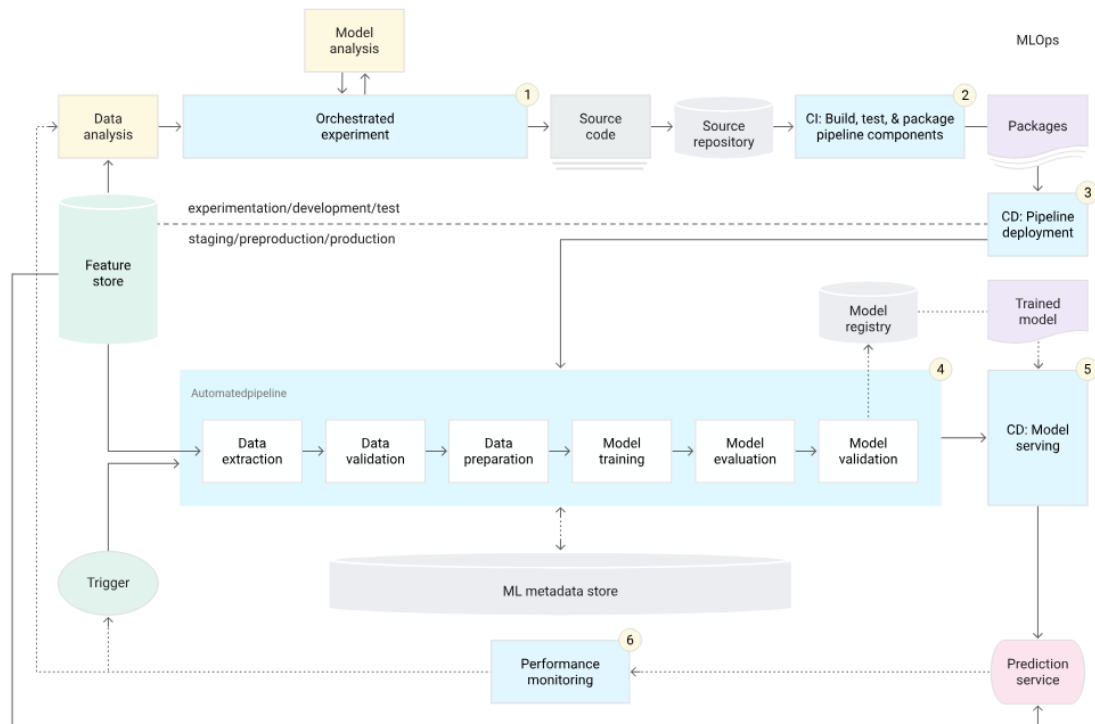


Figure 4. CI/CD and automated ML pipeline.

This MLOps setup includes the following components:

- Source control
- Test and build services
- Deployment services
- Model registry
- Feature store
- ML metadata store
- ML pipeline orchestrator

Characteristics

The following diagram shows the stages of the ML CI/CD automation pipeline:

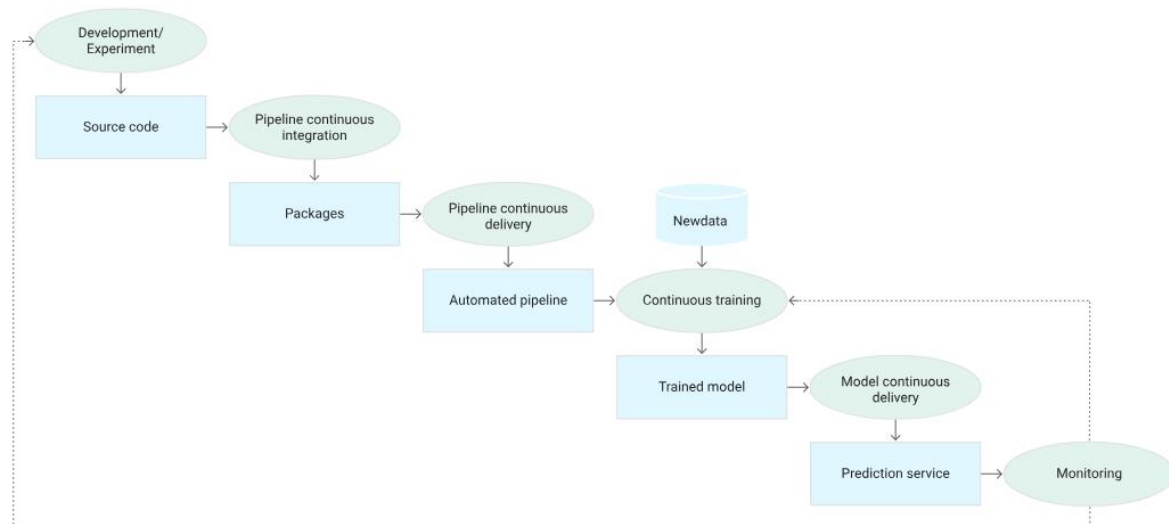


Figure 5. Stages of the CI/CD automated ML pipeline.

The pipeline consists of the following stages:

1. **Development and experimentation:** You iteratively try out new ML algorithms and new modeling where the experiment steps are orchestrated. The output of this stage is the source code of the ML pipeline steps that are then pushed to a source repository.
2. **Pipeline continuous integration:** You build source code and run various tests. The outputs of this stage are pipeline components (packages, executables, and artifacts) to be deployed in a later stage.
3. **Pipeline continuous delivery:** You deploy the artifacts produced by the CI stage to the target environment. The output of this stage is a deployed pipeline with the new implementation of the model.
4. **Automated triggering:** The pipeline is automatically executed in production based on a schedule or in response to a trigger. The output of this stage is a trained model that is pushed to the model registry.
5. **Model continuous delivery:** You serve the trained model as a prediction service for the predictions. The output of this stage is a deployed model prediction service.
6. **Monitoring:** You collect statistics on the model performance based on live data. The output of this stage is a trigger to execute the pipeline or to execute a new experiment cycle.

The data analysis step is still a manual process for data scientists before the pipeline starts a new iteration of the experiment. The model analysis step is also a manual process.

Continuous integration

In this setup, the pipeline and its components are built, tested, and packaged when new code is committed or pushed to the source code repository. Besides building packages, container images, and executables, the CI process can include the following tests:

- Unit testing your feature engineering logic.
- Unit testing the different methods implemented in your model. For example, you have a function that accepts a categorical data column and you encode the function as a [one-hot](#) feature.
- Testing that your model training converges (that is, the loss of your model goes down by iterations and [overfits](#) a few sample records).
- Testing that your model training doesn't produce [NaN](#) values due to dividing by zero or manipulating small or large values.
- Testing that each component in the pipeline produces the expected artifacts.
- Testing integration between pipeline components.

Continuous delivery

In this level, your system continuously delivers new pipeline implementations to the target environment that in turn delivers prediction services of the newly trained model. For rapid and reliable continuous delivery of pipelines and models, you should consider the following:

- Verifying the compatibility of the model with the target infrastructure before you deploy your model. For example, you need to verify that the packages that are required by the model are installed in the serving environment, and that the required memory, compute, and accelerator resources are available.
- Testing the prediction service by calling the service API with the expected inputs, and making sure that you get the response that you expect. This test usually captures problems that might occur when you update the model version and it expects a different input.
- Testing prediction service performance, which involves load testing the service to capture metrics such as [queries per second \(QPS\)](#) and model latency.
- Validating the data either for retraining or batch prediction.

- Verifying that models meet the predictive performance targets before they are deployed.
- Automated deployment to a test environment, for example, a deployment that is triggered by pushing code to the development branch.
- Semi-automated deployment to a pre-production environment, for example, a deployment that is triggered by merging code to the main branch after reviewers approve the changes.
- Manual deployment to a production environment after several successful runs of the pipeline on the pre-production environment.

To summarize, implementing ML in a production environment doesn't only mean deploying your model as an API for prediction. Rather, it means deploying an ML pipeline that can automate the retraining and deployment of new models. Setting up a CI/CD system lets you automatically test and deploy new pipeline implementations. This system lets you cope with rapid changes in your data and business environment. You don't have to immediately move all of your processes from one level to another. You can gradually implement these practices to help improve the automation of your ML system development and production.

What's next

- Learn more about [Architecture for MLOps using TensorFlow Extended, Gemini Enterprise Agent Platform Pipelines, and Cloud Build](#).
- Learn about the [Practitioners Guide to Machine Learning Operations \(MLOps\)](#).
- Watch the [MLOps Best Practices on Google Cloud \(Cloud Next '19\)](#) on YouTube.
- For an overview of architectural principles and recommendations that are specific to AI and ML workloads in Google Cloud, see the [AI and ML perspective](#) in the Well-Architected Framework.
- For more reference architectures, diagrams, and best practices, explore the [Cloud Architecture Center](#).

Contributors

Authors:

- [Jarek Kazmierczak](#) | Solutions Architect
- [Khalid Salama](#) | Staff Software Engineer, Machine Learning
- [Valentín Huerta](#) | AI Engineer

Other contributor: [Sunil Kumar Jang Bahadur](#) | Customer Engineer